

# Topology-Informed Visual Prompting for Hierarchical Vision-Language-Action Control

Haoyang Wu   Abhinav Kumar   Dmitry Berenson

Robotics Department, University of Michigan, Ann Arbor, MI, USA  
{haoyangw, abhin, dmitryb}@umich.edu

**Abstract:** Agentic robot systems often separate high-level reasoning from low-level visuomotor control, motivating the use of intermediate representations that communicate task-relevant constraints between layers. We study this interface in manipulation tasks where partial observations of obstacle geometry can obscure topological relationships that require qualitatively different actions. While motion planners with full knowledge of environment geometries and object states can reason about these signatures in planning, this information is often not known at deployment. To address this issue, we present a topology-guided visual-prompting framework that uses simulation-based planning to augment a nominal demonstration dataset and provides vision-based guidance at deployment. Our method uses a Gauss-Linking-Integral topological signature representation to capture important topological properties of the environment. Using privileged geometry information from a simulation approximation of our environment, we augment a VLA fine-tuning dataset with trajectories that move the system to a demonstrated signature and, from the new configuration, resume task execution. At deployment, a vision-language model (VLM) estimates the current signature and predicts image-space end-effector waypoints from camera observations. Recomputing both after each VLA action chunk yields a closed-loop hierarchical policy in which the VLM performs topological state estimation and waypoint prediction, while the VLA executes continuous control. Across three simulated bimanual tasks and a real-world box pickup task, our method outperforms a VLA fine-tuned only on nominal demonstrations and a VLM-prompting baseline that can remove topology-relevant information from observations. On hardware, it exceeds the strongest baseline by 40% in task success. Project website: <https://topology-vla.github.io>.

**Keywords:** Agentic Robotics, Visual Prompting, Vision-Language-Action Models

## 1 Introduction

Generalist robot policies [1, 2, 3] and VLA models [4, 5, 6, 7] are capable controllers, but typically map observations and instructions directly to actions without representing global robot–object–environment relationships. Agentic robot architectures address this limitation by separating task-level reasoning from continuous control and communicating intermediate representations such as programs, 3D value maps, language motions, or image-space trajectories [8, 9, 10, 11, 12]. We instantiate this interface with topology, which distinguishes geometrically similar configurations that require different motions. Specifically, we build on the h-signature, a loop-based representation that encodes how task-relevant loops are linked and allows planners with known geometry to constrain motion [13, 14]. Our agent extends this representation to partial observability by estimating topology from images, communicating corrective guidance to a VLA controller, and updating that guidance from feedback.

Rare topological mismatches are difficult to cover with demonstrations. We therefore use an approximate simulator to augment a nominal VLA dataset. The simulator need only preserve task-relevant topology and collisions, rather than appearance or exact geometry. Unlike generic dataset aggregation [15], topology identifies recovery states and correction objectives for generating targeted correction and task-continuation demonstrations.

At deployment, a separate VLM serves as the reasoning module because action training can degrade the reasoning abilities of VLA backbones [16, 17]. It infers the current topology and predicts correction waypoints, which are drawn over the observation as a visual prompt for the VLA controller [18, 19, 11, 20, 21]. Figure 1 shows how the prompt directs the robot either toward the goal or around an obstacle. After each action chunk, the VLM updates its estimate and prompt from the new observation, forming a closed-loop agentic hierarchy without modifying the VLA.

Because manipulation paths may be open before contact, we close them with *virtual links*, as shown in Fig. 1. We call the resulting quantity the virtual h-signature  $h^v$ , which connects data generation in simulation with visual reasoning at deployment.

We make two contributions.

- Topology-aware data generation uses  $h^v$  in approximate simulation to identify mismatches and generate targeted correction and task-continuation demonstrations.
- Closed-loop agentic control uses the same abstraction for VLM reasoning and communicates correction waypoints to a VLA through visual prompts that are updated after each action chunk.

Across the simulated tasks, our method remains competitive when no signature correction is required and improves success when execution requires transitions to different demonstrated  $h^v$  values. Ablations show that topology both structures the generation of targeted recovery data and provides a reasoning abstraction through which the VLM communicates state-dependent guidance to the VLA. On real-world Box Pickup, our method achieves 60% task success, exceeding the strongest baseline by 40 percentage points.

## 2 Preliminaries

We represent the task-relevant relationship between a robot-object loop  $L_t$  and obstacle loop  $O$  with the virtual h-signature  $h^v = \text{Lk}(L_t, O)$ , where  $\text{Lk}$  is the Gauss linking number, an integer measuring the signed winding between two closed loops [13]. We extend this quantity to open manipulation paths using virtual closing edges [14]. Because  $h^v$  changes only when the loops pass through one another, it distinguishes configurations that may look locally similar but require qualitatively different actions. Supplementary Appendix B gives the full construction.

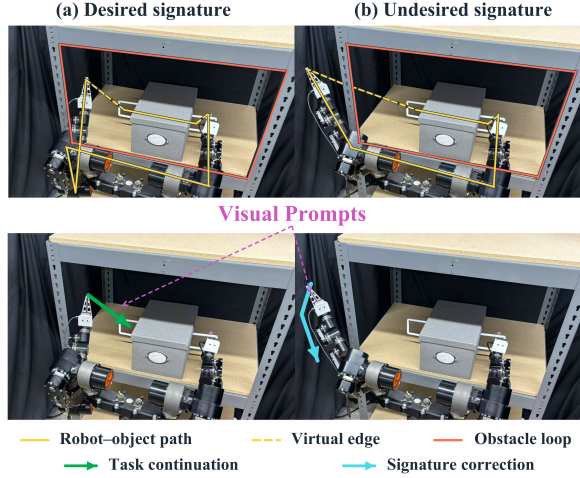


Figure 1: Different topological signatures require different motion directions for the same task goal. With the demonstrated signature, our green visual prompt guides the robot directly toward the handle. With an incorrect signature, our cyan prompt guides it around the rack post before approaching the handle.

### 3 Problem Statement

We consider manipulation tasks whose success depends on achieving a specific  $h^v$ . Let  $x_t \in \mathcal{X}$  denote the robot–object–environment configuration at time  $t$ , and let  $h_t^v := h^v(x_t)$  denote its  $h^v$ -signature at time  $t$ . We denote the demonstrated task signature by  $h^{v*} \in \mathbb{Z}$ . Because a mismatch  $h_t^v \neq h^{v*}$  can prevent direct progress toward the goal, successful execution may require first transitioning the system into the demonstrated signature. We assume that the task-relevant obstacle loop  $O$  is provided by a demonstrator. Given  $O$  and a demonstration, we can infer the demonstrated signature  $h^{v*}$ .

We assume access to a simulator of our tasks, in which we collect data. The simulator need not exactly reproduce the geometry and appearance of the deployment environment, instead serving as an approximate model for computing  $h^v$  values given  $O$ .  $O$  is only used to collect the simulation environment and is not assumed to be known in the real world. From the simulator, we collect an offline dataset

$$\mathcal{D} = \{\tau_i\}_{i=1}^N, \quad \tau_i = \{(o_t, s_t, h_t^v, a_t)\}_{t=0}^{T_i}.$$

Here,  $o_t$  is the visual observation,  $s_t$  is the robot state, and  $a_t$  denotes the expert action chunk beginning at time  $t$ . Each trajectory is also associated with a task instruction  $\ell$  and demonstrated  $h$ -signature  $h^{v*}$ . The task-relevant obstacle geometry remains fixed within each rollout and is visible in  $o_t$ . Given  $\mathcal{D}$ , our goal is to learn a manipulation policy that completes the desired task. To do this, we need a framework that detects whether the current  $h^v$  matches the demonstrated  $h^v$  and corrects a mismatch when needed.

At deployment,  $h_t^v$  is not observed. We assume that  $o_t$  contains sufficient visual information to infer  $h_t^v$ . Nevertheless, configurations with similar local geometry can have different  $h^v$  and require qualitatively different manipulation strategies. Therefore, geometric proximity alone may not determine how the robot should progress toward the task goal. We evaluate a policy by its closed-loop task success rate, which measures whether it resolves  $h^v$  mismatches when necessary and completes the manipulation task.

### 4 Method

A standard VLA maps visual observations, robot state, and a language instruction directly to robot motion. In our setting, this mapping is ambiguous because similar visual inputs under the same instruction can correspond to different signatures and require different motions. By augmenting the observation input to the VLA, our method resolves this ambiguity. Our method defines a hierarchical policy in which a VLM estimates  $h^v$  and generates waypoint prompts given the  $h^v$  estimate that guide a VLA. After each action chunk, both predictions are recomputed from the updated visual observation. Figure 2 summarizes this architecture. We describe topology-aware demonstration generation in Sec. 4.1, construction of the visual prompts in Sec. 4.2, and VLM training and closed-loop guidance in Sec. 4.3.

#### 4.1 $h^v$ -Aware Demonstration Generation

Our goal is to augment nominal demonstrations  $\mathcal{D}_{\text{nominal}}$  with a correction dataset  $\mathcal{D}_{\text{corr}}$  which moves configurations to the demonstrated  $h^v$ , and a continuation dataset  $\mathcal{D}_{\text{cont}}$ , which resumes task progress after correction, resulting in dataset

$$\mathcal{D}_{\text{train}} = \mathcal{D}_{\text{nominal}} \cup \mathcal{D}_{\text{corr}} \cup \mathcal{D}_{\text{cont}}. \quad (1)$$

We first train a base policy,  $\pi_{\text{base}}$ , by fine-tuning a pretrained VLA on a nominal demonstration dataset  $\mathcal{D}_{\text{nominal}}$ . These demonstrations cover successful task execution from randomized initial configurations, but they do not include explicit  $h^v$  annotations or targeted recovery behaviors for correcting  $h^v$  mismatches. As a result,  $\pi_{\text{base}}$  captures the nominal manipulation skills required for task completion, while leaving  $h^v$ -specific recovery behaviors underrepresented.

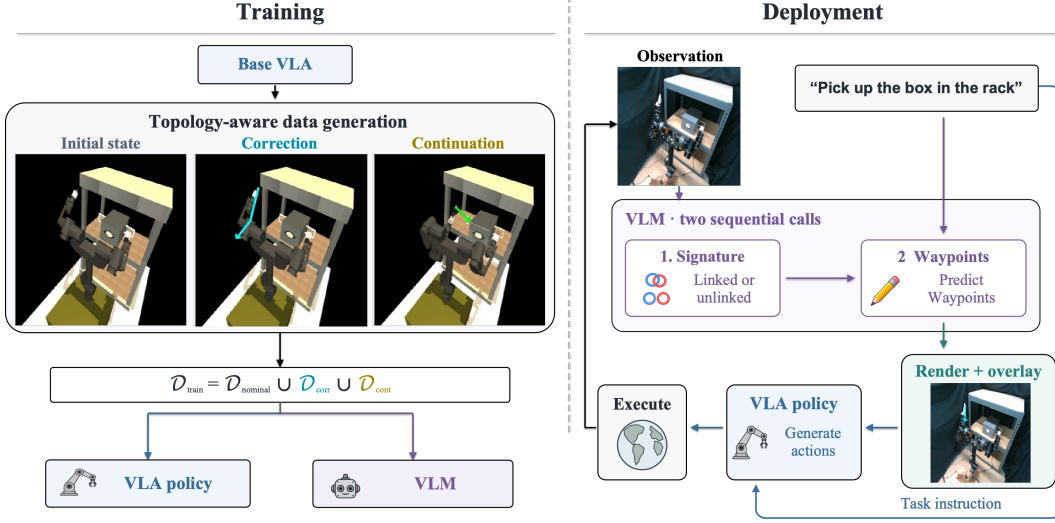


Figure 2: Overview of the training and deployment pipelines. During training,  $h^v$ -correction and task-continuation trajectories augment the nominal demonstrations used to supervise the VLA and VLM. At deployment, the VLM first predicts the current  $h^v$  and then predicts image-space waypoints. At each policy update, these waypoints are rendered on the current observation to guide the VLA’s next action chunk.

Being in the incorrect  $h^v$  could lead to a collision or block progress toward the goal. For example, as shown in Figure 3, a robot might move the cable tip toward the goal while leaving it outside the target loop. To augment the dataset with more diverse trajectories, we execute  $\pi_{\text{base}}$  in closed loop in simulation using the visual observation, robot state, and task instruction, and perturb states encountered along these rollouts. Privileged  $h^v$  labels from the simulator assign each perturbation resulting in a collision-free configuration to the appropriate recovery-data branch. The complete collection procedure is summarized in Algorithm 1.

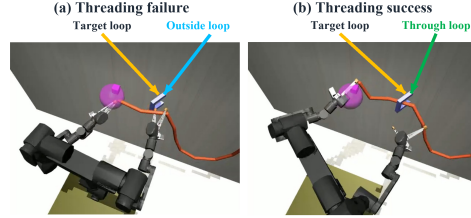


Figure 3: Successful and failed cable-threading rollouts. In (a), the cable passes through the target loop before reaching the purple goal. In (b),  $\pi_{\text{base}}$  moves the cable tip toward the goal while leaving it outside the loop.

At each collection step, PERTURBSTATE adds Gaussian noise to the current end-effector position, and a low-level controller moves the end effector toward the resulting target to produce  $\tilde{x}_t$ . This motion is used only to construct the perturbed configuration and is not in the training dataset. Unreachable targets are discarded. For each perturbation resulting in a collision-free configuration, the simulator computes  $h^v(\tilde{x}_t)$  and compares it with the demonstrated label  $h^{v*}$ .

When  $h^v(\tilde{x}_t) \neq h^{v*}$ , CORRECTSIGNATURE searches for a motion that is collision-free with respect to the physical robot, object, and environment and that changes the label to the demonstrated class. We sample candidate Cartesian targets around the active end effector and use the workspace-goal-directed RRT formulation of Vande Weghe et al. [22] to plan a collision-free joint-space trajectory to each target. Each resulting trajectory is executed in simulation and retained only if its terminal configuration  $x^+$  satisfies  $h(x^+) = h^{v*}$ . When multiple valid corrections are found, we select the trajectory with the shortest end-effector path length.

After CORRECTSIGNATURE changes a mismatched configuration to a corrected state  $x^+$ , this state may lie outside the distribution covered by the nominal demonstrations. CONTINUE TASK therefore executes  $\pi_{\text{base}}$  from  $x^+$  and checks whether it reaches the next task subgoal while maintaining

---

**Algorithm 1:**  $h^v$ -Aware Demonstration Generation

---

**Input:** Base VLA  $\pi_{\text{base}}$ , simulator  $\mathcal{E}$ , demonstrated h-signature  $h^{v*}$ **Output:** Correction data  $\mathcal{D}_{\text{corr}}$  and continuation data  $\mathcal{D}_{\text{cont}}$  $\mathcal{D}_{\text{corr}}, \mathcal{D}_{\text{cont}} \leftarrow \emptyset$ **foreach** task instance **do**    Initialize the task state  $x_0$     **while** the nominal rollout is active **do**        Execute  $\pi_{\text{base}}$  to obtain the next state  $x_t$          $\tilde{x}_t \leftarrow \text{PERTURBSTATE}(x_t)$         **if**  $\tilde{x}_t \neq \emptyset$  **then**            Compute  $h^v(\tilde{x}_t)$  using the simulator            **if**  $h^v(\tilde{x}_t) \neq h^{v*}$  **then**                 $\tau_{\text{corr}} \leftarrow \text{CORRECTSIGNATURE}(\tilde{x}_t, h^{v*})$                 **if**  $\tau_{\text{corr}} = \emptyset$  **then**                    **continue**                 $\mathcal{D}_{\text{corr}} \leftarrow \mathcal{D}_{\text{corr}} \cup \{\tau_{\text{corr}}\}$                  $\tau_{\text{cont}} \leftarrow \text{CONTINUETASK}(x^+, \pi_{\text{base}}, h^{v*})$                 **if**  $\tau_{\text{cont}} \neq \emptyset$  **then**                     $\mathcal{D}_{\text{cont}} \leftarrow \mathcal{D}_{\text{cont}} \cup \{\tau_{\text{cont}}\}$         **else**            **continue**

---

$h(x) = h^{v*}$ . If the base VLA succeeds, CONTINUETASK returns  $\emptyset$ , no continuation demonstration is added, and data collection proceeds to the next state of the nominal rollout. If the base VLA fails, a low-level end-effector IK controller generates a fallback continuation trajectory. We add only this fallback trajectory to  $\mathcal{D}_{\text{cont}}$ , provided that it reaches the subgoal while maintaining  $h(x) = h^{v*}$ . Otherwise, no continuation demonstration is added. If perturbation or correction fails, no demonstration is added for that candidate.

We train the final policy,  $\pi_{\text{VLA}}$ , as a separate fine-tuning run initialized from the original pretrained VLA rather than from  $\pi_{\text{base}}$ . All trajectories in  $\mathcal{D}_{\text{train}}$  are pooled without balancing or reweighting the three data sources. We describe how these end-effector trajectories are converted into visual prompts for VLM training below.

## 4.2 Visual Prompt Construction

We separate topology reasoning from action prediction because jointly adapting a VLM for reasoning and robot actions can degrade its pretrained reasoning capabilities [16, 17]. Following prior work on trajectory annotations [18, 19], we communicate  $h^v$  guidance through image-space drawings without modifying the VLA architecture.

We convert the end-effector trajectories generated in Sec. 4.1 into image-space drawings that are overlaid on the VLA’s visual observations. At each timestep, we project the desired end-effector motion onto the image from the scene camera. We denote the ordered image-space waypoint coordinates by  $W_t$ . A deterministic renderer connects these waypoints in temporal order and adds an arrowhead to indicate the direction of motion. We overlay this drawing on the visual observation  $o_t$  to obtain the prompted observation  $\tilde{o}_t$ .

We construct visual prompts for both  $h^v$ -correction and task-continuation trajectories.  $h^v$ -correction trajectories are rendered as arrows depicting the motion required to reach  $h^{v*}$ . These prompts consist of full trajectories and are capable of representing non-linear end-effector motion, for example a curved path around the post of a rack, by using multiple waypoints. Task-continuation trajectories are rendered as arrows in a different color from the topology-correction arrows, pointing toward the next task subgoal, such as a grasp location. Nominal task demonstrations are left unmodified. Each prompted observation  $\tilde{o}_t$  is paired with the expert action chunk  $a_t$ , allowing the VLA to learn how

Table 1: Simulation task-success rates (%). Base Policy is fine-tuned on  $\mathcal{D}_{\text{nominal}}$ .

| Task            | Method      | VLA backbone |                 |
|-----------------|-------------|--------------|-----------------|
|                 |             | $\pi_0$ [6]  | $\pi_{0.5}$ [7] |
| Rope Pulling    | Base Policy | <b>70</b>    | 52              |
|                 | PEEK [20]   | 54           | 50              |
|                 | Ours        | 66           | 64              |
| Cable Threading | Base Policy | 49           | 47              |
|                 | PEEK [20]   | 13           | 20              |
|                 | Ours        | 71           | <b>77</b>       |
| Box Pickup      | Base Policy | 31           | 45              |
|                 | PEEK [20]   | 57           | 66              |
|                 | Ours        | <b>84</b>    | 82              |

Table 2: Virtual  $h^v$ -signature recognition accuracy (%) on unseen configurations.

| Task            | VLA backbone |                 |
|-----------------|--------------|-----------------|
|                 | $\pi_0$ [6]  | $\pi_{0.5}$ [7] |
| Cable Threading | 74           | 76              |
| Box Pickup      | 92           | 92              |

image-space guidance maps to continuous control. The projected waypoint coordinates  $W_t$  also serve as supervision targets for the VLM described in the next subsection. Privileged quantities, including  $h_t^v$  and three-dimensional planning targets, are not provided to the policy.

### 4.3 Closed-Loop Hierarchical Guidance

We train the VLA using drawings constructed from privileged  $h^v$  labels and expert trajectories, which are unavailable at deployment. We therefore train a single VLM to recognize the current  $h^v$  from camera observations and then predict the image-space waypoint coordinates. We separate these tasks because  $h^v$  labels are easier to obtain than expert trajectories.

We train the VLM with two types of supervised examples: 1)  $h^v$ -prediction examples pair a visual observation  $o_t$  with its simulator-provided label  $h_t^v$ ; and 2) Waypoint-prediction examples pair  $(o_t, \ell, h_t^v, h^{v*})$  with the ordered image-space coordinates  $W_t$  constructed in Sec. 4.2. Here,  $h_t^v$  provides the ground-truth current signature during training, and  $h^{v*}$  specifies the demonstrated signature. For nominal training states, the waypoint target is an empty sequence.

At deployment, we query the same VLM twice. We denote the  $h^v$  and waypoint calls by  $f_{h^v}$  and  $f_{\text{way}}$ . The first call predicts the virtual h-signature from the visual observation,

$$\hat{h}_t^v = f_{h^v}(o_t).$$

Because the ground-truth current label is unavailable at deployment, the second call uses  $\hat{h}_t^v$  with the visual observation, task instruction, and known demonstrated signature  $h^{v*}$ ,

$$\hat{W}_t = f_{\text{way}}(o_t, \ell, \hat{h}_t^v, h^{v*}).$$

The output is an ordered sequence of image-space waypoint coordinates. We render these coordinates as an arrow using the same visual conventions employed during VLA training, then overlay the resulting transparent layer on  $o_t$  to obtain  $\tilde{o}_t$ . An empty waypoint sequence leaves the observation unchanged. The VLA then maps the prompted observation, robot state, and original task instruction to a continuous action chunk,

$$\hat{a}_{t:t+H_a-1} = \pi_{\text{VLA}}(\tilde{o}_t, s_t, \ell),$$

where  $H_a$  is the action-chunk horizon. After executing each predicted chunk, we acquire a new observation and repeat both VLM calls. Recomputing the signature and waypoints from each observation updates the guidance as the task progresses and allows subsequent predictions to revise earlier errors.

Table 3: Real-world Box Pickup success rates (%). Base Policy is fine-tuned on  $\mathcal{D}_{\text{nominal}}$ .

| Method                  | Overall task success (%) | $h^v$ -correction success (%) |
|-------------------------|--------------------------|-------------------------------|
| Base Policy $\pi_0$ [6] | 20                       | 30                            |
| PEEK [20]               | 3                        | 20                            |
| Ours                    | 60                       | 83                            |

## 5 Results

Our experiments address four questions. Does our method improve task performance across different  $h^{v*}$  values and VLA backbones, remain effective when only one  $h^v$  is possible, and transfer to real hardware? We evaluate Cable Threading and Box Pickup as tasks with multiple possible  $h^v$  values, Rope Pulling as a task with only a single possible  $h^v$ , and Box Pickup on hardware to assess real-world feasibility.

We evaluate Rope Pulling (no signature change), Cable Threading (correction to  $h^{v*} = 1$ ), and Box Pickup (correction to  $h^{v*} = 0$ ) across two VLA backbones, with 50, 100, and 100 simulation trials, respectively, and 30 Box Pickup hardware trials. Full task definitions, evaluation protocols, and training details appear in Supplementary Appendices C and D.

### 5.1 Simulation Results

#### 5.1.1 Quantitative Results

We use the same baselines and evaluation metrics for all three simulated tasks. For each VLA backbone,  $\pi_0$  [6] and  $\pi_{0.5}$  [7], we compare the base policy trained without data augmentation or additional annotations, PEEK [20], and our  $h^v$ -guided visual prompting. Table 1 reports task success rates for each method and backbone.

On Rope Pulling, our method performs comparably to the base policies and exceeds PEEK [20] by 12–14% across backbones, demonstrating compatibility with tasks with no signature correction. PEEK’s lower performance is likely because Rope Pulling repeatedly alternates arms that grasp and pull the rope, making it hard to separate the end effectors’ interleaved motions into subtrajectories.

On Cable Threading, our method exceeds the base policies by 22–30% and PEEK [20] by 57–58% across the two backbones. These gains support the proposed pipeline for execution requiring a transition to  $h^{v*} = 1$ . As in Rope Pulling, PEEK again struggles to distinguish the two arms’ subtrajectories during alternating motions. Here, execution must additionally respect the relationship between the robot–object and obstacle loops, making correct motion selection essential.

On Box Pickup, our method exceeds the base policies by 37–53% and PEEK [20] by 16–27% across the two backbones. Our method retains an advantage even when PEEK’s prompts benefit execution. Together with Cable Threading, these results support the pipeline’s applicability to tasks requiring different demonstrated signatures, including correction to  $h^{v*} = 0$ , rather than only a single type of transition.

We next examine whether the VLM recovers the topological state needed to generate appropriate guidance. We evaluate signature recognition independently of policy execution on 100 randomly sampled unseen configurations per task. As shown in Table 2, the VLM recognizes the signature with 74–76% accuracy on Cable Threading and 92% accuracy on Box Pickup. The higher accuracy on Box Pickup mirrors its higher end-to-end success, suggesting that signature estimation contributes to task performance.

Ablations using the  $\pi_{0.5}$  backbone test the two roles of topology in our framework. First, removing VLM-generated prompts while retaining the same topology-aware training data decreases success by 12, 8, and 7 percentage points on Rope Pulling, Cable Threading, and Box Pickup, respectively. This result shows that topology is useful not only to guide data generation, but also as a reasoning

abstraction for communicating state-dependent guidance from the VLM to the VLA at execution time. Second, the task-continuation-only ablation disables topology-aware data generation: the generator no longer uses signature mismatches to identify recovery states or to specify correction objectives, and augments the nominal dataset only with task-continuation trajectories. Although this intervention necessarily removes the resulting correction trajectories, it more fundamentally prevents topology from determining what recovery data should be generated. The corresponding 53-point drop on Cable Threading and 28-point drop on Box Pickup show that topology provides a useful abstraction for directing data generation toward failures that nominal task-continuation data do not address. Together, these results establish topology as both a data-generation abstraction and an interface between high-level visual reasoning and continuous control (Supplementary Appendix E). Qualitative examples in Supplementary Appendix G further illustrate how this interface preserves obstacle context and revises incorrect intermediate predictions.

## 5.2 Real-World Results

To account for sim-to-real transfer, we adapt all VLAs with 18 short real-world trajectories and the VLM with 1,000 signature-labeled observations; Supplementary Appendix F gives the protocol.

### 5.2.1 Quantitative Results

We report both overall task success and  $h^v$ -correction success.  $h^v$ -correction success measures whether the policy moves the robot from a configuration with an initial incorrect signature to a final configuration with the demonstrated signature. Under the real-world adaptation protocol described above, our method exceeds the strongest baseline by 40–53% on both metrics, as shown in Table 3. The improvement is therefore not limited to signature correction but extends to task completion on hardware. For every method, correction success exceeds overall task success, showing that reaching the demonstrated signature does not by itself ensure a successful lift.

Qualitatively, the VLM can revise an incorrect signature prediction at a subsequent observation and redirect the VLA around the rack post, indicating that repeated state estimation and waypoint prediction can recover from intermediate errors (Supplementary Appendix G).

## 6 Conclusion

We presented a topology-guided hierarchical framework that combines simulation-based data generation, VLM-based state estimation and waypoint prediction, and VLA control. Privileged simulator geometry supports the generation of signature-correction and task-continuation demonstrations for VLA fine-tuning. At deployment, a fine-tuned VLM predicts task-relevant signatures and end-effector waypoints, providing visual guidance without requiring full environment geometry. Across three simulated bimanual tasks and real-world box pickup, our method outperforms a VLM-prompting baseline and improves hardware task success by 40% over the strongest baseline. Our ablations show that topology is useful in two distinct capacities: as an abstraction for identifying and generating targeted recovery data, and as an interface for communicating high-level visual reasoning to continuous control. Closed-loop prompt updates further allow the policy to revise intermediate prediction errors.

## References

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [2] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *ICRA*, 2024.
- [3] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [4] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al.  $\pi_0$ : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [7] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky.  $\pi_{0.5}$ : a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>.
- [8] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation*, 2023.
- [9] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. In *Conference on Robot Learning*, 2023.
- [10] S. Belkhale, T. Ding, T. Xiao, P. Sermanet, Q. Vuong, J. Tompson, Y. Chebotar, D. Dwibedi, and D. Sadigh. RT-H: Action hierarchies using language. In *Robotics: Science and Systems*, 2024.
- [11] Y. Li, Y. Deng, J. Zhang, J. Jang, M. Memmel, C. Garrett, F. Ramos, D. Fox, A. Li, A. Gupta, et al. Hamster: Hierarchical action models for open-world robot manipulation. In *ICLR*, 2025.
- [12] L. X. Shi, B. Ichter, M. R. Equi, L. Ke, K. Pertsch, Q. Vuong, J. Tanner, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. In *International Conference on Machine Learning*, 2025.
- [13] S. Bhattacharya, M. Likhachev, and V. Kumar. Topological constraints in search-based robot path planning. *Autonomous Robots*, 2012.
- [14] P. Mitrano and D. Berenson. The grasp loop signature: A topological representation for manipulation planning with ropes and cables. In *ICRA*, 2024.
- [15] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011.

- [16] A. J. Hancock, X. Wu, L. Zha, O. Russakovsky, and A. Majumdar. Actions as language: Fine-tuning vlms into vlas without catastrophic forgetting. *arXiv preprint arXiv:2509.22195*, 2025.
- [17] S. Yang, H. Li, B. Wang, Y. Chen, Y. Tian, T. Wang, H. Wang, F. Zhao, Y. Liao, and J. Pang. Instructvla: Vision-language-action instruction tuning from understanding to manipulation. *arXiv preprint arXiv:2507.17520*, 2025.
- [18] J. Gu, S. Kirmani, P. Wohlhart, Y. Lu, M. G. Arenas, K. Rao, W. Yu, C. Fu, K. Gopalakrishnan, Z. Xu, et al. Rt-trajectory: Robotic task generalization via hindsight trajectory sketches. *arXiv preprint arXiv:2311.01977*, 2023.
- [19] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *ICLR*, 2025.
- [20] J. Zhang, M. Memmel, K. Kim, D. Fox, J. Thomason, F. Ramos, E. Bıyık, A. Gupta, and A. Li. Peek: Guiding and minimal image representations for zero-shot generalization of robot manipulation policies. *arXiv preprint arXiv:2509.18282*, 2025.
- [21] S. Nasiriany, F. Xia, W. Yu, T. Xiao, J. Liang, I. Dasgupta, A. Xie, D. Driess, A. Wahid, Z. Xu, et al. Pivot: Iterative visual prompting elicits actionable knowledge for vlms. *arXiv preprint arXiv:2402.07872*, 2024.
- [22] M. Vande Weghe, D. Ferguson, and S. S. Srinivasa. Randomized path planning for redundant manipulators without inverse kinematics. In *Humanoids*, 2007.